

DOCUMENTO DE EVALUACIÓN · CONTROL DE CALIDAD

Rúbrica QA RESERVAYA

Evaluación del análisis y las pruebas de los módulos de Registro de usuario e Inicio de sesión.

Versión evaluada: primera versión funcional

Fuente de referencia: requerimientos del sistema y registro docente de defectos

Resultado esperado: identificar, reproducir, categorizar y documentar los 10 bugs intencionales

Documento preparado para revisión del equipo QA.

1. Objetivo del equipo QA

Evaluar que los módulos de registro e inicio de sesión cumplan los requerimientos funcionales y las reglas de negocio definidas, mediante pruebas de interfaz, API, sesión y persistencia de datos.

El equipo debe entregar evidencia suficiente para que otra persona pueda entender el defecto, repetirlo y confirmar el resultado sin depender de explicaciones verbales.

2. Alcance

Módulo 1 Registro de usuario: campos obligatorios, formato de correo, contraseña, confirmación, unicidad y respuesta exitosa.	Módulo 2 Inicio de sesión: credenciales válidas/incorrectas, mensajes, sensibilidad del correo y cierre de sesión.
API /api/registro, /api/login, /api/logout y /api/me.	Datos Verificación en PostgreSQL/pgAdmin de duplicados, espacios, nombres y contraseñas protegidas.

Fuera de alcance: reservas, pagos y panel de administración; no forman parte de esta versión.

3. Categorización de problemas

Categoría	Qué se evalúa	Bug relacionados
Validación y límites	Reglas obligatorias, longitud mínima y formato válido.	BUG-01, BUG-08, BUG-09
Integridad de datos	Unicidad y normalización de valores antes de comparar o guardar.	BUG-02, BUG-03
Reglas en backend	El servidor debe aplicar las reglas aunque se omita la interfaz.	BUG-04, BUG-08
Concurrencia y UX	Control de envíos repetidos y consistencia de mensajes.	BUG-05
Autenticación y seguridad	Mensajes no reveladores, acceso con correo normalizado y protección post-cierre.	BUG-06, BUG-07, BUG-10

4. Pruebas realizadas

Tipo	Propósito	Ejemplos
Funcionales UI	Comprobar flujos normales y validaciones visibles.	Registro válido, campos vacíos, contraseña de 8 caracteres, login correcto/incorrecto.
Límites y negativos	Explorar valores frontera y entradas inválidas.	8 caracteres, espacios, correo con mayúsculas y correo con espacio interno.
API / caja negra	Verificar que el backend no dependa de la validación del navegador.	POST /api/registro sin confirmación o con nombre compuesto por espacios.
Persistencia	Confirmar el efecto real en la base de datos.	Duplicados por capitalización/espacios y valores almacenados en usuarios.
Sesión y navegación	Comprobar el estado después de cerrar sesión.	Logout y botón Atrás del navegador; consulta de sesión activa.
Repetición / concurrencia	Detectar solicitudes duplicadas y respuestas contradictorias.	Doble clic o varios Enter en Crear cuenta.

5. Rúbrica de evaluación

La calificación sugerida es de **100 puntos**. Se considera aprobado el trabajo que demuestra evidencia reproducible y cubre los defectos que correspondan al alcance.

Criterio	Indicador de logro	Puntos
Objetivo como equipo QA	Define el propósito de las pruebas y relaciona calidad con requerimientos y reglas de negocio.	10
Alcance	Incluye registro, login, sesión, API y BD; distingue lo incluido de reservas/pagos/administración.	10
Categorización	Clasifica los problemas por validación, datos, backend, UX, autenticación y seguridad de forma coherente.	10
Pruebas realizadas	Describe casos positivos, negativos, límites, API, persistencia, repetición y navegación.	15
Correspondencia de bugs	Identifica correctamente los 10 bugs intencionales y los relaciona con el módulo/requerimiento afectado.	20
Evidencia clara	Entrega capturas, respuestas HTTP, datos de BD o registros de red con contexto, fecha/caso y resultado.	15
Pasos para reproducir	Cada reporte contiene precondiciones, datos, pasos numerados, resultado esperado y resultado actual.	10
Conclusión QA	Resume riesgos, severidad/prioridad, impacto y recomendación de corrección o regresión.	10

6. Matriz de bugs esperados

ID / módulo	Problema y requerimiento	Sev.	Prior.	Evidencia mínima
BUG-01 Registro	Contraseña de exactamente 8 caracteres rechazada por límite incorrecto. REQ-REG-007	Media	Alta	Captura del formulario y respuesta 400 del servidor.
BUG-02 Registro	Correo duplicado con mayúsculas se acepta. REQ-REG-005	Alta	Alta	Solicitud exitosa y dos filas equivalentes en BD.
BUG-03 Registro	Espacio al inicio/final del correo no se elimina y permite otra cuenta. REQ-REG-005	Media	Media	Valor con espacio visible en consulta de BD.
BUG-04 Registro / API	Backend no valida confirmación de contraseña ausente o diferente. REQ-REG-008	Alta	Alta	POST reproducible, código 201 y cuenta creada.
BUG-05 Registro	Dobles envíos generan solicitudes y mensajes contradictorios.	Baja-Media	Media	Registro de red con solicitudes repetidas.
BUG-06 Login	Mensajes distintos permiten saber si el correo existe.	Media	Media	Comparación de dos respuestas 401.
BUG-07 Login	Correo con mayúsculas no permite login con contraseña correcta.	Media	Media	Respuesta de cuenta inexistente y credenciales usadas.
BUG-08 Registro / API	Nombre o apellido compuesto solo por espacios es aceptado.	Media	Media	POST exitoso y campo de espacios en BD.
BUG-09 Registro / Login	Correo con espacio interno pasa el formato y se almacena.	Baja-Media	Media	Valor inválido aceptado y registro en BD.
BUG-10 Login / Sesión	Tras logout, Atrás puede mostrar contenido de bienvenida desde bfcache.	Alta	Alta	Video/capturas de navegación y estado de sesión 401.

7. Casos de prueba y reproducción

CP-01 · Límite de contraseña

Datos: usuario válido y Abcd1234 en ambos campos.

Pasos: abrir Registro → completar datos → enviar → observar respuesta.

Esperado: cuenta creada. **Actual a reportar:** rechazo por mínimo de 8 caracteres. Evidencia: captura y respuesta HTTP.

CP-02 · Duplicidad normalizada

Precondición: existe usuario1@reservaya.cl. Registrar Usuario1@reservaya.cl y luego un correo con espacio final.

Esperado: rechazo por correo existente y normalización. **Actual:** cuentas distintas o valor con espacio persistido. Evidencia: consulta de usuarios.

CP-03 · Validaciones solo en frontend

Datos: POST directo a /api/registro con confirmPassword diferente/ausente y con nombre: " ".

Esperado: HTTP 400. **Actual:** registro aceptado. Evidencia: cuerpo de solicitud/respuesta y fila creada.

CP-04 · Envío repetido

Pasos: completar registro válido → doble clic rápido o pulsar Enter varias veces → revisar Network y alertas.

Esperado: una sola operación controlada. **Actual:** solicitudes múltiples y posible contradicción de mensajes.

CP-05 · Mensajes y capitalización en login

Datos: correo inexistente; luego usuario1@reservaya.cl con contraseña errónea; finalmente USUARIO1@RESERVAYA.CL con Reserva123.

Esperado: mensaje genérico y acceso con correo equivalente. **Actual:** mensajes diferenciados y rechazo por capitalización.

CP-06 · Protección tras logout

Pasos: iniciar sesión → llegar a bienvenida → cerrar sesión → usar Atrás → observar pantalla y consultar sesión.

Esperado: no mostrar contenido autenticado. **Actual:** contenido puede quedar visible brevemente por bfcache. Evidencia: video/capturas con navegador y resultado.

8. Evidencia y decisión QA

Un reporte se considera bien sustentado cuando incluye: ID único, módulo, requerimiento, entorno/navegador, precondiciones, datos de prueba, pasos numerados, resultado esperado, resultado actual, severidad, prioridad y una evidencia verificable.

Los bugs BUG-01 a BUG-10 corresponden al alcance de esta evaluación porque afectan directamente Registro, Inicio de sesión, API, persistencia o control de sesión. BUG-06 y BUG-10 son hallazgos de seguridad/experiencia derivados de buenas prácticas y del requisito de acceso con sesión, aunque no todos sus detalles estén redactados literalmente en los requerimientos.

Conclusión QA: la versión no debería considerarse lista para aprobación sin corregir primero BUG-02, BUG-04 y BUG-10 por su impacto en unicidad, validación del servidor y exposición de contenido tras el cierre de sesión. Luego se recomienda ejecutar regresión completa de los casos CP-01 a CP-06 y verificar que las reglas se apliquen tanto en UI como en API.